

Esercizio A1

Andando per tentativi, proviamo con $\Theta(n\sqrt{n})$. È facile vedere che la ricorrenza è $\Omega(n\sqrt{n})$, per via della sua componente non ricorsiva. Proviamo quindi a dimostrare che $T(n) = O(n\sqrt{n})$.

- Caso base: $T(n) = 1 \leq cn\sqrt{n}$, per tutti i valori di n compresi fra 1 e 9, ovvero:

$$c \geq \frac{1}{n\sqrt{n}}, \forall n : 1 \leq n \leq 9$$

I valori $\frac{1}{n\sqrt{n}}$ sono minori o uguali di 1, per $1 \leq n \leq 9$; quindi tutte queste disequazioni sono soddisfatte da $c \geq 1$.

- Ipotesi induttiva: $T(k) \leq ck\sqrt{k}$, per $k < n$
- Passo induttivo:

$$\begin{aligned} T(n) &= 4T(\lfloor n/4 \rfloor) + 9T(\lfloor n/9 \rfloor) + n\sqrt{n} \\ &\leq 4c\lfloor n/4 \rfloor^{1.5} + 9c\lfloor n/9 \rfloor^{1.5} + n\sqrt{n} \\ &\leq \frac{4}{8}cn\sqrt{n} + \frac{9}{27}cn\sqrt{n} + n\sqrt{n} \\ &= \frac{1}{2}cn\sqrt{n} + \frac{1}{3}cn\sqrt{n} + n\sqrt{n} \\ &\leq 5/6cn\sqrt{n} + n\sqrt{n} \leq cn\sqrt{n} \end{aligned}$$

L'ultima disequazione è vera per $c \geq 6$.

Abbiamo quindi dimostrato che $T(n) = \Theta(n\sqrt{n})$, con $m = 1$ e $c \geq 6$.

Esercizio A2

La soluzione si basa (ovviamente) sulla ricerca binaria, ma bisogna prestare particolare attenzione ai casi particolari e agli indici. In particolare, se il valore v è più alto di tutti i valori presenti nel vettore, il valore cercato non esiste e bisogna restituire -1 .

- **Caso base:** Si consideri un vettore costituito da un elemento solo.
 - Se tale vettore contiene un valore più piccolo o uguale a v , significa che non esiste un elemento più grande di v nel vettore e dobbiamo restituire -1 .
 - Se tale vettore contiene un valore più grande di v , è anche il più piccolo con questa proprietà presente nel vettore e quindi è il valore da restituire.
- **Passo ricorsivo:** Si considera l'elemento mediano in posizione $m = \lfloor (i + j)/2 \rfloor$.
 - Se v è maggiore o uguale a $A[m]$, allora il valore cercato, se esiste, si trova in $A[m + 1 \dots j]$.
 - Altrimenti, se v è più minore $A[m]$ il valore esiste sicuramente, e si trova in $A[i \dots m]$ (essendo potenzialmente $A[m]$ stesso).

Ricalcando la ricerca binaria, l'algoritmo ha una complessità pari a $O(\log n)$, come richiesto.

```
int ceilRec(int [] A, int i, int j, int v)
```

```
    if i == j then
        if v ≥ A[i] then
            return -1
        else
            return A[i]
    else
        int m = ⌊(i + j)/2⌋
        if v ≥ A[m] then
            return ceilRec(A, m + 1, j, v)
        else
            return ceilRec(A, i, m, v)
```

```
int ceil(int [] A, int n, int v)
```

```
    return ceilRec(A, 1, n, v)
```

Esercizio A3

L'esercizio si risolve tramite una visita in ampiezza dell'albero binario, e ed è molto simile al problema di calcolare la larghezza dell'albero radicato (Problema 1.1 del blocco di esercizi sugli Alberi e Problema 5.4 del libro). La soluzione fornita per quell'esercizio deve essere modificata per due aspetti:

- L'albero che prendiamo in input è binario, non generale, quindi va modificata la parte che inserisce nodi in coda.
- Invece di calcolare la larghezza, calcoliamo la somma delle brillantezze, che poi va confrontata con la brillantezza della radice.

Il costo dell'algoritmo è ovviamente $\Theta(n)$.

boolean isChristmassy(**TREE** *t*)

```
int count = 1                                % # nodi nel livello corrente da visitare; radice
int brillianceSoFar = 0                      % Brillantezza del livello attuale
boolean christmassy = true                 % Vero se l'albero rispetta la definizione
QUEUE Q = Queue()
Q.enqueue(t)
while not Q.isEmpty() and christmassy do
    TREE u = Q.dequeue()
    if u.left()  $\neq$  nil then
        | Q.enqueue(u.left())
    if u.right()  $\neq$  nil then
        | Q.enqueue(u.right())
    brillianceSoFar = brillianceSoFar + u.brilliance
    count = count - 1
    if count == 0 then                        % Finito livello
        | count = Q.size()
        | if brillianceSoFar  $\neq$  t.brilliance then
            | | christmassy = false
            | | brillianceSoFar = 0
    |
return christmassy
```
